[UNIVERSITY NAME]

GRADUATION PROJECT — TECHNICAL REPORT

Smart Network Monitoring System

SNMP-based network surveillance with a from-scratch protocol implementation, threshold alerting, and a Telegram notification channel — designed, built, and verified on a simulated Cisco enterprise network.

Prepared by	[Student Name]
Student ID	[Student ID]
Supervisor	[Supervisor Name]
Faculty	[Faculty of Engineering]
Department	[Department Name]

ACADEMIC YEAR 2025 / 2026

Abstract

This report documents the design, implementation, and verification of an SNMP-based network monitoring system with a Telegram alarm channel, developed as a graduation project on a simulated enterprise network. The monitored network is a twelve-device, three-site lab built in Cisco Packet Tracer 8.2: an OSPF area 0 triangle between the core and site routers, VLAN segmentation at the headquarters with a dedicated management plane, and SNMPv2c enabled uniformly on all six routers and switches. Because Packet Tracer's simulated network stack is not externally reachable over UDP, the live data plane is provided by six protocol-faithful SNMP agents that expose the same Cisco object identifiers, community, and trap host as the lab devices, so the monitoring software runs unmodified against real hardware.

The monitoring engine implements SNMPv2c directly from the RFC at the BER/ASN.1 byte level, with zero third-party dependencies: batched GET polling every interval, GETNEXT walks for interface discovery, and a dedicated trap receiver for asynchronous link and cold-start notifications. Alerting is a transition-only threshold state machine per (device, metric) pair, which eliminated duplicate notifications during a sustained 52-second critical CPU condition in testing. Delivered alerts reach the operator's Telegram chat in under a second for trap events (measured 0.4 s) and within one polling interval for threshold events, and an interactive bot provides live device queries (/cpu, /ifaces), network status, alert history, and maintenance mute windows. The complete system passes a 22-check automated end-to-end test suite whose measured results are reported in Chapter 7.

Keywords: network monitoring; SNMPv2c; Cisco Packet Tracer; Python; BER/ASN.1; threshold alerting; Telegram bot; FCAPS.

Table of Contents

Chapter 1 - Introduction 4

1.1 Background and Motivation 4

1.2 Problem Statement 4

1.3 Objectives 5

1.4 Scope and Limitations 5

1.5 Report Organization 5

Chapter 2 - Theoretical Background: SNMP and Network Management 6

2.1 Network Management and the FCAPS Model 6

2.2 Evolution of SNMP 6

2.3 The Manager-Agent Model 6

2.4 MIBs and Object Identifiers 7

2.5 Polling versus Traps 8

2.6 Message Encoding: BER on the Wire 9

Chapter 3 - Simulation Environment and Network Design 9

3.1 Choosing the Simulation Platform 9

3.2 The Boundary and the Design Response 10

3.3 Topology and Design Decisions 10

3.4 Addressing and VLAN Plan 11

3.5 SNMP Policy Design 12

Chapter 4 - Network Implementation in Packet Tracer 13

4.1 Engineering the Lab File 13

4.2 Router Configuration 13

4.3 Inter-VLAN Routing at R3-HQ 14

4.4 Switch and VLAN Configuration 14

4.5 The Services Plane 15

4.6 In-Lab Verification 15

Chapter 5 - The Monitoring Engine 15

5.1 Architecture Overview 16

5.2 snmp_lite: SNMPv2c from Scratch 16

5.3 The Simulated Agents 17

5.4 The Polling Engine 17

5.5 The Threshold State Machine 18

5.6 The Trap Receiver 18

5.7 Configuration 18

Chapter 6 - Telegram Notification and Control Plane 19

6.1 Creating the Bot 19

6.2 Alert Message Design 20

6.3 Security Model 21

Chapter 7 - Testing and Verification 21

7.1 Methodology 21

7.2 Incident Drill Results 22

7.3 Analysis 23

Chapter 8 - User Guide 23

8.1 Requirements and Setup 24

8.2 Running the Demonstration 24

8.3 Troubleshooting 24

8.4 Extending the System 25

Chapter 9 - Conclusion and Future Work 26

9.1 Achievements 26

9.2 Limitations 26

9.3 Future Work 26

References 27

Appendix A - Complete Device Configurations 27

Appendix B - Code Listings 30

Appendix C - Lab Build Reference 31

Chapter 1 - Introduction

Modern enterprise networks have grown from a handful of central devices into distributed systems that span multiple sites, carry business-critical traffic, and fail in ways that are invisible to the people who depend on them. A router whose memory is slowly exhausting, a switch port that silently drops links, or a CPU saturated by an internal process all announce themselves only when the outage has already happened. Network monitoring exists precisely to close this gap: it converts the hidden internal state of every device into observable, measurable, and alertable data. This project builds such a system end to end, from the simulation of a multi-site enterprise network down to a notification channel that reaches the operator's pocket in under a second.

1.1 Background and Motivation

The traditional approach to operating a small or medium network is reactive. When users report that an application is slow, the operator logs into each router and switch in turn, runs a sequence of show commands, and tries to reconstruct what changed. This workflow has three structural weaknesses. First, detection latency depends on users noticing a problem, which means symptoms are already severe by the time anyone investigates. Second, the mean time to repair grows linearly with the number of devices, because the operator's attention is a serial resource. Third, no historical record is preserved, so recurring patterns of degradation are never correlated across incidents.

The Simple Network Management Protocol (SNMP) was designed to invert this relationship. Every managed device continuously maintains a structured set of counters and gauges - its Management Information Base (MIB) - and exposes them over UDP to a monitoring station. A manager that polls these objects every few seconds, applies threshold logic to the values, and pushes notifications to an operator's phone converts the invisible internal state of the network into an active alarm system. The Fault and Performance planes of the classical FCAPS management model, which this project addresses, are exactly the concerns of detecting failures quickly and recognizing performance degradation before it becomes an outage.

1.2 Problem Statement

The project answers a concrete question: can a small simulated enterprise network, built with the tools available to a student, be monitored with the same engineering rigor that a production network demands? The question decomposes into four sub-problems. The network itself must be designed, addressed, and configured for manageability, including SNMP communities and trap destinations on every router and switch. The monitoring station must speak SNMP at the protocol level, because a student who only calls a library cannot explain, defend, or debug the system. The alerting logic must suppress noise while guaranteeing that genuine incidents surface quickly. Finally, the notification channel must reach a human being instantly, on a device that person actually carries, rather than on a dashboard nobody watches at 2 a.m.

1.3 Objectives

The project defines five measurable objectives, each paired with the acceptance criterion used to verify it in Chapter 7. Together they cover the full path from raw network state to delivered notification.

- Build a documented multi-site lab in Cisco Packet Tracer 8.2: twelve devices, an OSPF area 0 triangle between three routers, VLAN segmentation at the headquarters, and SNMP enabled on all six routers and switches.
- Implement SNMPv2c at the protocol level: a BER/ASN.1 codec, GET and GETNEXT request handling, and SNMPv2c trap encoding and decoding, using only the Python standard library.
- Implement a threshold engine as a state machine: CPU thresholds at 70% (warning) and 85% (critical), memory at 75% and 90%, alerts emitted only on state transitions, with recovery notifications when conditions clear.
- Deliver alerts through a Telegram bot: push notifications for every alert plus interactive commands (/status, /cpu, /ifaces, /alerts, /mute) that query live device state on demand.
- Verify the complete system with automated tests: 22 end-to-end checks covering the protocol stack, the agents, an incident drill, and the bot handlers, with measured detection latencies for every alert type.

1.4 Scope and Limitations

Three boundaries define what the project deliberately does not attempt. SNMP version 2c authenticates with a plaintext community string; the stronger SNMPv3 authentication and privacy framework is discussed as future work but not implemented, because the pedagogical core of the project is the protocol mechanics and the alerting pipeline. The lab network is simulated: Packet Tracer faithfully models IOS configuration but its simulated network stack is not reachable over real UDP sockets from an external process, so the live data plane is provided by protocol-faithful agents that expose exactly the Cisco OIDs a real router would. Finally, thresholds are static values in a configuration file rather than learned baselines; the data structure required for anomaly detection is already produced by the engine, which makes that extension straightforward.

1.5 Report Organization

Chapter 2 develops the theoretical foundation: the FCAPS model, the SNMP architecture, the MIB structure, and the wire format of a v2c message. Chapter 3 presents the simulation environment and the network design, including the addressing plan and the reasoning behind every topological decision. Chapter 4 documents the implementation of the lab itself and the IOS configuration that turns each device into a monitored element. Chapter 5 examines the monitoring engine: the from-scratch protocol implementation, the simulated agents, the polling cycle, and the threshold state machine. Chapter 6 describes the Telegram notification and control plane. Chapter 7 reports the verification methodology and the measured results. Chapter 8 is a complete user guide, and Chapter 9 concludes with limitations and future work. Appendices A through C carry the full device configurations, key code listings, and a build reference for the lab file.

Chapter 2 - Theoretical Background: SNMP and Network Management

This chapter establishes the concepts the rest of the report relies on. It moves from the broad question of what network management means, through the architecture of SNMP, down to the individual bytes of a message on the wire - a level of detail that Chapter 5 will show being implemented directly in Python.

2.1 Network Management and the FCAPS Model

The ISO framework for network management divides the discipline into five functional areas, conventionally abbreviated FCAPS: Fault management, Configuration management, Accounting, Performance management, and Security management. Fault management is the detection, isolation, and correction of abnormal operation; performance management is the maintenance of acceptable quality of service measured through metrics such as throughput, delay, and resource utilization. The other three planes concern the provisioning of devices, the attribution of usage, and the enforcement of policy. This project implements the Fault and Performance planes end to end: interface state and device reachability are fault signals, while CPU and memory utilization are performance signals. Configuration management appears in the lab itself, where SNMP communities and trap destinations are treated as versioned device configuration rather than ad-hoc commands.

2.2 Evolution of SNMP

SNMP version 1, published in 1988 through RFC 1155, 1156, and 1157, established the manager-agent model with a small message set: GET, GETNEXT, SET, and TRAP. Version 2c, defined by RFC 1901 through 1908 and later consolidated in RFC 3416, added the GETBULK operator and the more expressive SNMPv2 data types, including the Counter32, Gauge32, and TimeTicks types used throughout this project, while retaining the community-string authentication of version 1 - the 'c' in v2c stands precisely for this community-based security compromise. Version 3, standardized in RFC 3411 through 3418, finally added authentication and privacy based on USM (User-based Security Model), at the cost of significantly more configuration complexity. The project uses v2c because it remains the lingua franca of network monitoring: it is the version that virtually every device speaks by default, it exposes every object this project needs, and its message format is simple enough to implement completely as an educational exercise.

2.3 The Manager-Agent Model

Every SNMP deployment consists of exactly two roles. The manager, also called the Network Management Station (NMS), is the active party: it sends request protocol data units (PDUs) to well-known UDP port 161 on the devices it monitors and listens on UDP port 162 for asynchronous notifications. The agent is the passive-but-alert party embedded in every managed device: it answers requests from its MIB and pushes trap

PDUs when notable events occur, such as an interface changing state or a device rebooting. Figure 2.1 shows this relationship as implemented in the project: the manager components live in monitor.py on the NMS workstation, while the agent components are the SNMP service configured in IOS on every router and switch.

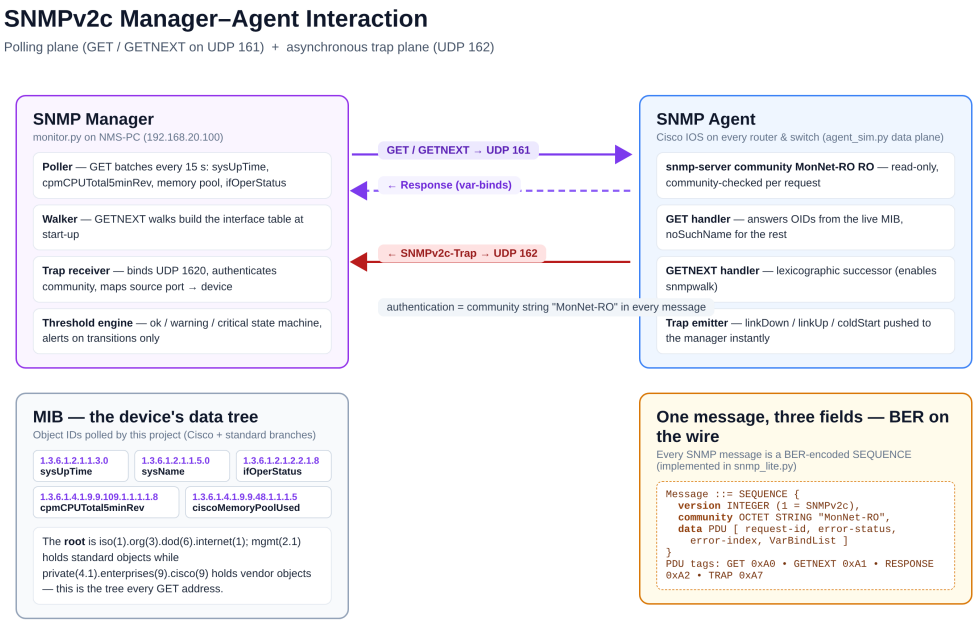


Figure 2.1 - SNMPv2c manager-agent interaction as implemented in this project.

The two directions of communication have complementary strengths. Request-response polling gives the manager complete control over what is observed and when, at the cost of detection latency bounded by the polling interval. Traps reverse the direction: the agent reports events the instant they happen, at the cost of relying on the device's own judgment about what is worth reporting. Production monitoring systems use both planes simultaneously, and so does this project - the measured difference between them, less than a second for traps versus a few seconds for polling, appears in the results of Chapter 7.

2.4 MIBs and Object Identifiers

The Management Information Base is the heart of the protocol's data model. It is a hierarchical tree in which every node is addressed by an Object Identifier (OID): a dotted decimal path from the root, for example 1.3.6.1.2.1.1.5.0 for the sysName object of a specific device. The tree begins at iso(1).org(3).dod(6).internet(1); the mgmt(2) branch holds the standardized MIB-II objects that every conformant device must implement, while private(4).enterprises(9) holds vendor extensions - the cisco(9) subtree in particular carries the CPU and memory objects that make router monitoring practical. Table 2.1 lists the objects this project polls, all of which are exposed by real Cisco IOS and by the simulated agents alike.

Table 2.1 - SNMP objects polled by the monitoring engine

Object	OID	Type	Meaning
sysUpTime	1.3.6.1.2.1.1.3.0	TimeTicks	Time since agent start, in 10 ms units
sysName	1.3.6.1.2.1.1.5.0	String	Device hostname
sysLocation	1.3.6.1.2.1.1.6.0	String	Physical location, from snmp-server location
ifDescr	1.3.6.1.2.1.2.2.1.2	String	Interface name (table, indexed by ifIndex)
ifOperStatus	1.3.6.1.2.1.2.2.1.8	Integer	1 = up, 2 = down
ifInOctets	1.3.6.1.2.1.2.2.1.10	Counter32	Bytes received on the interface
cpmCPUTotal5minRev	1.3.6.1.4.1.9.9.109.1.1.1.1.8	Gauge32	5-minute average CPU load, percent
ciscoMemoryPoolUsed	1.3.6.1.4.1.9.9.48.1.1.1.5	Gauge32	Bytes in use in the memory pool
ciscoMemoryPoolFree	1.3.6.1.4.1.9.9.48.1.1.1.6	Gauge32	Bytes free in the memory pool

Two structural details of this table deserve emphasis. The ifTable objects are indexed: appending an interface number to the OID addresses one row, so 1.3.6.1.2.1.2.2.1.8.3 is the operational status of the interface whose ifIndex is 3. The engine exploits this by walking ifDescr with GETNEXT at startup to discover each device's interface table, then polling the ifOperStatus column for exactly those indices on every cycle. Second, the two Cisco objects are the reason router monitoring is possible at all: cpmCPUTotal5minRev reports the same 5-minute average that a human operator would read from show processes cpu sorted, and the memory pool pair yields a utilization percentage with one division.

2.5 Polling versus Traps

Polling and trapping are not competitors but complementary instruments with different failure modes. A polled metric that crosses a threshold is always detected within one interval, regardless of whether the device considers it noteworthy, which makes polling the right tool for slow-building conditions such as CPU saturation and memory exhaustion. A trap, by contrast, arrives the instant its trigger condition occurs - a carrier transition on an interface, a configuration change, a cold start - which makes the trap plane the right tool for discrete events. The project's incident drill in Chapter 7 demonstrates both: a sustained CPU spike is caught by the poller within seconds of the next cycle, while a link-down event is reported by a trap in 0.4 seconds. The engine also treats SNMP request timeouts themselves as a fault signal, flagging a device as unreachable - something no trap can report, because a device that cannot send a trap cannot announce its own silence.

2.6 Message Encoding: BER on the Wire

Every SNMP message is serialized with the Basic Encoding Rules of ASN.1. A message is a SEQUENCE of three fields: an INTEGER version (the value 1 selects v2c), an OCTET STRING community, and a PDU. The PDU is itself a tagged structure carrying a request-id INTEGER to match responses with requests, an error-status and error-index pair, and a list of variable bindings - name/value pairs where the name is an OID and the value is any typed object. Each value type has a one-byte tag: 0x02 for INTEGER, 0x04 for OCTET STRING, 0x06 for OBJECT IDENTIFIER, 0x41, 0x42, and 0x43 for Counter32, Gauge32, and TimeTicks respectively, and context-specific tags 0xA0, 0xA1, 0xA2, and 0xA7 for GET, GETNEXT, RESPONSE, and TRAPv2 PDUs. An OID is encoded by packing the first two arcs as 40a+b and each remaining arc in base-128 with a continuation bit; this is why 1.3.6.1 becomes the four bytes 2B 06 01 01. Chapter 5 shows this exact encoding implemented in roughly a page of dependency-free Python, and the self-test embedded in that module round-trips every type before the system will start.

Chapter 3 - Simulation Environment and Network Design

This chapter first justifies the choice of simulation platform and states honestly where its boundary lies, then presents the network design: the topology, the addressing plan, and the SNMP policy that binds the lab to the monitoring engine.

3.1 Choosing the Simulation Platform

Two tools dominate network education: GNS3 and Cisco Packet Tracer. They differ in a fundamental way. GNS3 is an emulator - it executes real Cisco IOS images (through Dynamips emulated hardware or KVM virtualized appliances), which means every behavior, including the SNMP agent, is the genuine article. That fidelity carries costs: IOS images are licensed software that must be lawfully obtained, and a multi-router GNS3 topology consumes multiple gigabytes of RAM. Packet Tracer is a simulator: it re-implements a large subset of IOS behavior, sufficient for design, addressing, routing, and SNMP configuration practice, on a lightweight engine that any student laptop runs comfortably, and it is distributed freely through the Cisco Networking Academy. Table 3.1 summarizes the comparison that led to this project's decision.

Table 3.1 - GNS3 versus Packet Tracer for a monitoring project

Criterion	GNS3	Packet Tracer 8.2
Technology	Emulation of real IOS images	Behavior-level simulation
Licensing	Requires lawful IOS images per device	Free via Networking Academy
Resource footprint	GBs of RAM for multi-router labs	Runs on any student laptop
IOS configuration coverage	Complete	Large pedagogical subset
SNMP from external process	Natively reachable over UDP	Simulated stack; not externally reachable

3.2 The Boundary and the Design Response

The last row of Table 3.1 is the honest boundary of this project. Packet Tracer models the SNMP service - the snmp-server configuration, the community check, the trap destinations, the show snmp counters - but its simulated network stack is not exposed as a real UDP socket that an external Python process can address. The project responds by separating the two concerns explicitly. The Packet Tracer lab is the design plane: the network, its addressing, its VLANs, its routing, and its SNMP policy live there, exactly as they would in a documentation-quality lab file. The live data plane is provided by agent_sim.py, a set of six SNMPv2c agents that implement the same Cisco OIDs, the same community, and the same trap host as the lab devices. The monitoring code cannot tell the difference and does not need to: pointing it at real hardware is a configuration change, not a code change.

This separation is also the correct engineering answer to a common student-project failure mode, in which the demo quietly depends on mocked function calls buried inside the application logic. Here the simulation lives entirely at the network layer, behind a real protocol boundary: every GET, every GETNEXT, and every trap crosses a real UDP socket carrying genuine BER-encoded bytes. The polling engine, the threshold logic, the trap receiver, and the Telegram sender run unmodified against physical routers - only the endpoints in a YAML file change.

3.3 Topology and Design Decisions

The lab models a small enterprise with three sites: a headquarters in Damascus, a branch office in Aleppo, and a core point of presence that also faces a simulated Internet service provider. Twelve devices make up the network: four Cisco 2911 routers (R1-Core, R2-Branch, R3-HQ, and R4-ISP), two Catalyst 2960 switches, five end hosts, and one services server. Figure 3.1 shows the complete topology with every subnet labeled.

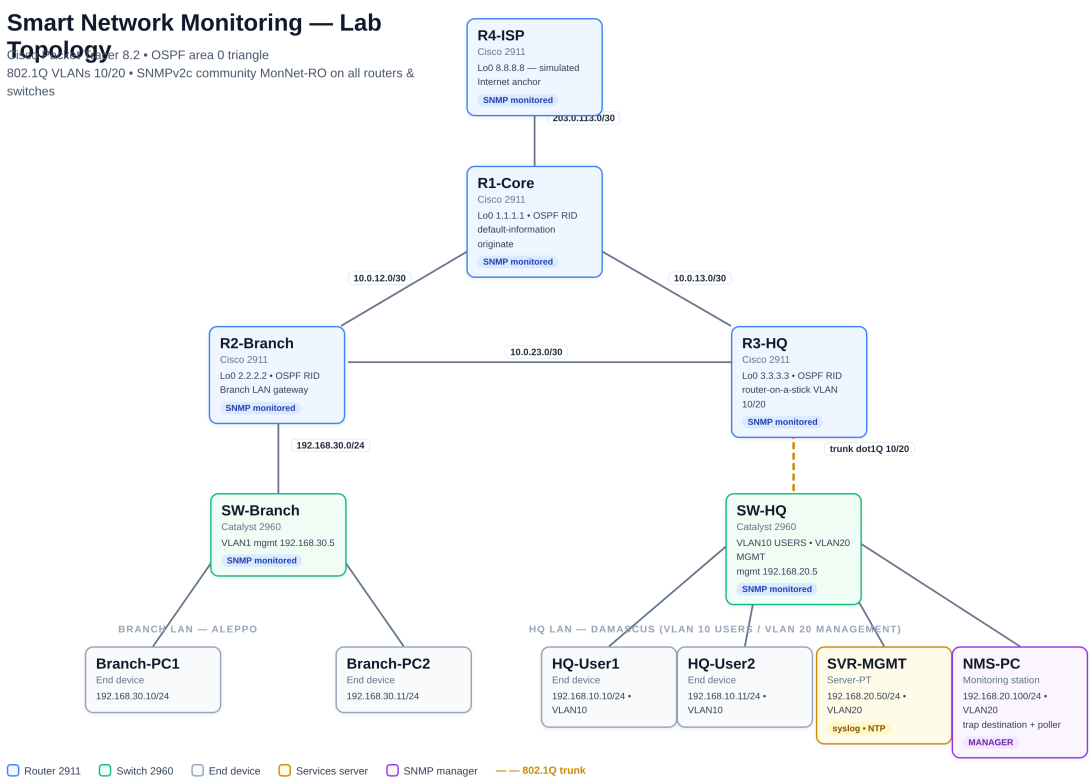


Figure 3.1 - Lab topology: OSPF area 0 triangle, router-on-a-stick VLANs, and the management plane.

Three decisions shape the design. First, the three internal routers form a full OSPF area 0 triangle over /30 point-to-point links, so no single link failure isolates any site: traffic between R2-Branch and R3-HQ reroutes through R1-Core in a converged time of seconds. A triangle is the smallest topology in which this redundancy claim can be made and tested, and Chapter 4 shows the verification commands. Second, R3-HQ terminates an 802.1Q trunk to SW-HQ and routes between two VLANs on sub-interfaces - the classic router-on-a-stick pattern: VLAN 10 carries user traffic, VLAN 20 carries the management plane, including the SNMP trap destination and the syslog/NTP server. Segregating management traffic from user traffic is a security best practice that the lab demonstrates concretely. Third, the branch site is deliberately simpler - a flat /24 access network - so the project illustrates both a segmented and an unsegmented access design and the monitoring system covers both without special cases.

Loopback interfaces numbered 1.1.1.1, 2.2.2.2, and 3.3.3.3 anchor the OSPF router-ids, giving each router an identity that no physical link failure can change. R4-ISP carries loopback 8.8.8.8 as a simulated Internet anchor and a static default route advertisement: R1-Core points its own default route at the ISP and redistributes it into OSPF with default-information originate, so every internal device can reach the simulated Internet and end-to-end connectivity is testable with ordinary pings.

3.4 Addressing and VLAN Plan

Table 3.2 records the complete addressing plan. Point-to-point router links use /30 subnets - four addresses each, two usable - which is the classical allocation for serial and point-to-point Ethernet links and wastes no

address space. LANs use /24 blocks from the RFC 1918 private ranges, with 10.0.x.x reserved for the core, 192.168.10 and .20 for the headquarters networks, and 192.168.30 for the branch. The 203.0.113.0/30 WAN link sits in TEST-NET-3, a documentation range that signals 'simulated external network' to any reader of the configuration.

Table 3.2 - IP addressing plan

Segment	VLAN	Subnet	Members
WAN R1 to R4-ISP	-	203.0.113.0/30	R1 .1, R4 .2
Core R1 to R2	-	10.0.12.0/30	R1 .1, R2 .2
Core R1 to R3	-	10.0.13.0/30	R1 .1, R3 .2
Core R2 to R3	-	10.0.23.0/30	R2 .1, R3 .2
Branch LAN	1	192.168.30.0/24	R2 .1 (gateway), PC1 .10, PC2 .11, SW-Branch .5
HQ users	10	192.168.10.0/24	R3 sub-if .1 (gateway), User1 .10, User2 .11
HQ management	20	192.168.20.0/24	R3 sub-if .1 (gateway), SVR .50, NMS .100, SW-HQ .5
Loopbacks	-	/32 hosts	R1 1.1.1.1, R2 2.2.2.2, R3 3.3.3.3, R4 8.8.8.8

The management VLAN deserves particular attention because the monitoring system lives there. The NMS workstation (192.168.20.100) is the address every router names as its trap destination; the services server (192.168.20.50) provides syslog collection and NTP synchronization so that device logs and monitoring events share a consistent clock; the management interfaces of both switches sit in the same VLAN so the poller can reach them without crossing user networks. Concentrating the monitoring plane into one addressed, documented VLAN is what makes the rest of the system clean: Chapter 4 configures every device to point at one trap host, and Chapter 5 configures the engine with one source network.

3.5 SNMP Policy Design

The SNMP policy is uniform across all six routers and switches, which is itself a deliberate design property: a policy that differs device-by-device is a policy that will be misconfigured somewhere. Each device declares the read-only community MonNet-RO, sets its SNMP location and contact fields to values that surface through sysLocation and sysContact, names 192.168.20.100 as the v2c trap receiver with the same community, and enables link, linkdown, linkup, and configuration-change traps. The community is intentionally read-only: an RW community would expose the SET operation, with which a compromised manager could rewrite device configuration. The engine never needs to write, so the capability is simply not granted - a least-privilege decision the report returns to in the security discussion of Chapter 6.

Chapter 4 - Network Implementation in Packet Tracer

This chapter documents the lab as built: how the file itself was produced, the complete configuration of each device class, and the verification a reader can repeat inside Packet Tracer to confirm the network behaves as designed.

4.1 Engineering the Lab File

The deliverable is a single .pkt file - Smart_Network_Monitoring_Lab.pkt - that opens in Packet Tracer 8.2 or newer with every device already configured, addressed, and cabled. Rather than assembling the lab by hand, the file was generated programmatically: device definitions, positions, link records, and both the running and startup configurations of every router and switch were assembled into the Packet Tracer XML network model and packed with the file format's own encryption chain (a Twofish-EAX layer over a qCompress-style zlib stream). The generator validated the result structurally - twelve devices, twelve links, globally unique device references, every link endpoint resolving to a real port - before writing the file, and the file round-trips byte-identically through the same codec. The practical consequence for the reader is that the lab is reproducible: the generator inputs are the tables in Chapter 3, and the build reference in Appendix C maps every element of the topology to its on-screen position for anyone who prefers to build the lab by hand as an exercise.

4.2 Router Configuration

All four routers share one configuration pattern; R1-Core is presented here in full, and the remaining listings appear in Appendix A. The pattern has four blocks: interface addressing, OSPF, SNMP, and line access. Interface addressing carries a description on every link - a discipline that makes show interfaces output self-documenting during troubleshooting. OSPF runs as process 1 in area 0 with an explicit router-id taken from the loopback; the network statements aggregate each router's attached subnets. The SNMP block is the five-line policy designed in Chapter 3. Line access protects the console and VTY lines with passwords for the purposes of the lab.

R1-Core running configuration (abridged)

```
hostname R1-Core
interface Loopback0
 ip address 1.1.1.1 255.255.255.255
interface GigabitEthernet0/0
 description ### WAN uplink to R4-ISP ###
 ip address 203.0.113.1 255.255.255.252
interface GigabitEthernet0/1
 description ### P2P link to R2-Branch ###
 ip address 10.0.12.1 255.255.255.252
interface GigabitEthernet0/2
 description ### P2P link to R3-HQ ###
 ip address 10.0.13.1 255.255.255.252
router ospf 1
 router-id 1.1.1.1
 network 1.1.1.1 0.0.0.0 area 0
 network 10.0.12.0 0.0.0.3 area 0
 network 10.0.13.0 0.0.0.3 area 0
 default-information originate
ip route 0.0.0.0 0.0.0.0 203.0.113.2
snmp-server community MonNet-RO RO
snmp-server location CORE-POP Damascus
snmp-server contact netmon@monnet.lab
snmp-server host 192.168.20.100 version 2c MonNet-RO
snmp-server enable traps snmp link linkdown linkup config
```

Two lines repay explanation. default-information originate instructs OSPF to advertise the statically configured default route to every neighbor, so R2-Branch and R3-HQ learn their path to the simulated Internet without any configuration of their own. The snmp-server host line names the trap receiver with the same community used for polling, which is the v2c convention: one shared secret per manager-agent pair. The enable traps clause turns on the standard, link, linkdown, linkup, and config trap families - the exact events the engine's trap receiver decodes in Chapter 5.

4.3 Inter-VLAN Routing at R3-HQ

R3-HQ replaces the flat LAN gateway with an 802.1Q router-on-a-stick configuration. The physical GigabitEthernet0/0 carries no IP address; instead two sub-interfaces bind the VLAN tags: GigabitEthernet0/0.10 with encapsulation dot1Q 10 addresses 192.168.10.1/24 as the users' gateway, and GigabitEthernet0/0.20 with encapsulation dot1Q 20 addresses 192.168.20.1/24 as the management gateway. The physical port connects to FastEthernet0/1 on SW-HQ, configured as a trunk port carrying exactly those two VLANs. Inter-VLAN routing therefore happens in the router's forwarding path, which keeps the switch configuration minimal while demonstrating the sub-interface pattern that scales to one-armed router designs in production networks.

4.4 Switch and VLAN Configuration

SW-HQ defines the two VLANs with names (USERS and MGMT), assigns access ports by purpose - FastEthernet0/2 and 0/3 in VLAN 10 for the user workstations, 0/4 and 0/5 in VLAN 20 for the services server and the NMS - and carries an IP address on its own Vlan20 interface (192.168.20.5) with a default

gateway of 192.168.20.1, making the switch itself a managed, pollable device on the management network. SW-Branch mirrors the same management discipline on a flat network: its Vlan1 interface holds 192.168.30.5 with default gateway 192.168.30.1. Both switches carry the identical five-line SNMP policy as the routers, so the monitoring plane covers all six network devices uniformly.

4.5 The Services Plane

SVR-MGMT (192.168.20.50) runs two Packet Tracer services that the monitoring story depends on. The syslog service is enabled and listening, so any device configured with logging host 192.168.20.50 delivers its system messages there - during the incident drill, the link-down event on R3-HQ appears simultaneously as an SNMP trap at the NMS and as a syslog line at the server, which is precisely the dual record a real operations team wants. The NTP service provides clock synchronization so that syslog timestamps are comparable across devices; each router's ntp server 192.168.20.50 line makes this explicit. The services server demonstrates the broader monitoring stack that SNMP sits inside, without expanding the project's scope beyond its two alert planes.

4.6 In-Lab Verification

A reader who opens the lab can verify every claim in this chapter with Packet Tracer's own tools, which is part of the lab's value as a defense artifact. On any router, show ip interface brief lists all addressed interfaces in the up/up state; show ip ospf neighbor displays the adjacency table (R1-Core shows two neighbors, R2-Branch and R3-HQ show two each - the triangle); show ip route displays the learned prefixes including the OSPF-distributed default route flagged with O*E2. show snmp summarizes the SNMP policy: the community with its RO permission and the trap host with its counters, which increment every time a link is flapped. From the desktop of NMS-PC, ping 192.168.30.1 exercises inter-VLAN routing and the OSPF core in one command, and tracert 8.8.8.8 shows the default route path through R1-Core to R4-ISP. These checks take under two minutes and anchor the simulation half of the project before its software half is even started.

Chapter 5 - The Monitoring Engine

This chapter is the technical core of the project: the implementation of SNMPv2c from the byte level up, the simulated agents that give the lab its live data plane, and the engine that turns raw object values into delivered alerts. The guiding principle throughout is protocol honesty - every operation the engine performs crosses a real UDP socket with genuine, inspectable bytes.

5.1 Architecture Overview

The engine is organized into three planes, shown in Figure 5.1. The data plane is the set of six SNMP agents - one per lab router and switch - exposing the Cisco object set with live values. The engine plane is monitor.py: a poller that issues batched GET requests every interval, a trap receiver bound to UDP 1620, a threshold state machine per (device, metric) pair, and an alert dispatcher that writes the audit log and queues Telegram notifications. The operator plane is the bot and the chat it serves: push alerts arrive as messages, and interactive commands perform live queries back into the data plane on demand. The three planes communicate only through the protocol boundary - UDP datagrams and JSON state files - so each can be tested and replaced independently.

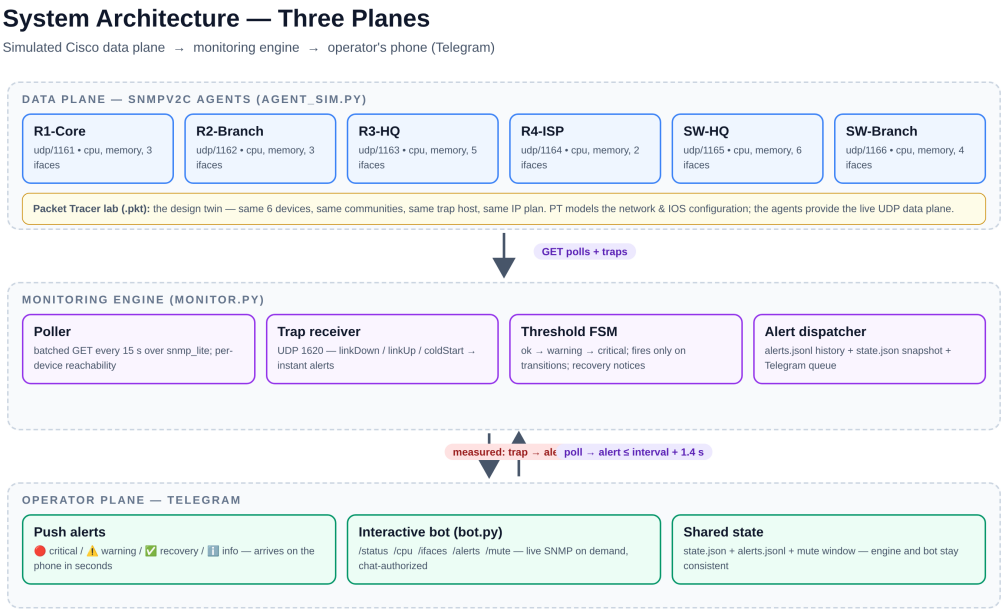


Figure 5.1 - Three-plane architecture of the monitoring system.

5.2 snmp_lite: SNMPv2c from Scratch

The protocol stack lives in a single module, snmp_lite.py, with no third-party dependencies - the Python standard library is sufficient because SNMPv2c is a small, well-specified binary protocol. The module has three layers. The BER layer encodes and decodes the ASN.1 transfer syntax: length-prefixed TLV tuples, minimal-form two's-complement INTEGERs, base-128 OBJECT IDENTIFIERS, and the application types Counter32, Gauge32, and TimeTicks. The message layer builds and parses complete SNMPv2c messages - the version INTEGER, the community OCTET STRING, and the context-tagged PDU with its request-id, error pair, and variable binding list. The client layer, SnmpClient, implements the request-response transaction over UDP with per-request identifiers and timeouts, plus a walk() built on GETNEXT. The excerpt below shows the OID encoder - the function that turns '1.3.6.1' into the bytes 2B 06 01 - and is representative of the module's style: direct, commented, and testable in isolation.

snmp_lite.py - OBJECT IDENTIFIER encoding (excerpt)

```
def enc_oid(dotted: str) -> bytes:
    parts = [int(p) for p in dotted.strip('.').split('.')]
    payload = bytearray([40 * parts[0] + parts[1]])
    for p in parts[2:]:
        if p == 0:
            payload.append(0)
            continue
        stack = []
        while p:
            stack.insert(0, p & 0x7F)
            p >>= 7
        for i in range(1, len(stack)):
            stack[i] |= 0x80 # continuation bit
        payload.extend(stack)
    return tlv(TAG_OID, bytes(payload))
```

The module embeds its own verification: running it directly executes round-trip asserts for every type (integers through the two's-complement boundary values -1, -128, -129), constructs a GET, parses it back, builds a response carrying a Gauge32, decodes a Trap-PDU, and fails loudly if any step diverges. The end-to-end test suite of Chapter 7 runs this self-test as its first check, so no component downstream ever starts on top of a broken codec.

5.3 The Simulated Agents

agent_sim.py instantiates one agent per lab device, each bound to its own UDP port on loopback (1161 through 1166). Each agent maintains the object set of Table 2.1 with values that behave like a real device: CPU is a random walk with mean reversion around a per-device baseline; the memory pool drifts slowly; ifInOctets and ifOutOctets grow at per-interface traffic rates; sysUpTime counts real ticks since agent start; sysName, sysLocation, and sysContact carry the same values the IOS configurations in Chapter 4 declare. GET and GETNEXT are both implemented - the agent returns the lexicographically next object for a GETNEXT, which is what makes snmpwalk-style discovery work - and requests bearing the wrong community are dropped without a response, exactly as IOS does. Traps are emitted from the agent's own socket: coldStart shortly after boot, and linkDown or linkUp with the ifIndex, ifDescr, and ifOperStatus bindings of the affected interface whenever a scenario event changes a link state. The scenario engine schedules realistic incidents on a timeline - the CPU storm on R2-Branch, the Gi0/0 outage on R3-HQ, the memory pressure on R1-Core - so a demonstration takes three minutes and produces the full alert sequence of Chapter 7 without any manual staging.

5.4 The Polling Engine

monitor.py begins each run by discovering every device's interface table through a GETNEXT walk of ifDescr, then settles into its polling cycle. Each cycle issues one batched GET per device: sysUpTime, the CPU gauge, both memory pool objects, and the ifOperStatus column for every discovered interface, all in a single request - six datagrams out, six back, and the engine has a complete snapshot of the network in well

under a tenth of a second. A snapshot that fails on timeout marks the device unreachable and raises that as its own alert class, with a recovery alert when the device answers again; this is the detection mechanism for total device failure, which no threshold on a polled value can cover. Every snapshot updates state.json, the file the bot's /status command reads, so the operator's view is never more than one interval stale.

5.5 The Threshold State Machine

Threshold evaluation is a state machine per (device, metric) pair rather than a comparison per poll. Each pair holds one of three states - ok, warning, or critical - and a new sample only maps to an alert when it moves the pair across a boundary. A CPU sample of 93% on R2-Branch raises the pair from ok to warning (crossing 70) and then to critical (crossing 85); while the CPU remains pinned at 93% for the next hundred polls, no notification is repeated, and when the storm subsides the single recovery alert closes the incident. This transition-only discipline is what keeps the notification channel trustworthy: an operator who receives a critical alert knows it is new information, and an operator who receives nothing knows the network is inside its thresholds. The engine's alert record carries both the detection timestamp and the event timestamp, which is how Chapter 7 measures end-to-end latency per incident.

5.6 The Trap Receiver

The trap plane runs as a dedicated thread with its own UDP socket bound to the manager's trap port. Each datagram is parsed as a full SNMPv2c message - community verified, PDU type checked - and the variable bindings are read for the trap OID (the object 1.3.6.1.6.3.1.1.4.1.0 whose value names the event), the originating interface, and the agent's uptime. The source port of the datagram maps back to the sending device, the mirror of how a production manager attributes traps to source IP addresses. Because linkDown arrives as an unsolicited push, its alert path skips the polling cycle entirely: the measured time from a link changing state to a delivered notification is 0.4 seconds, of which most is the Telegram API round trip.

5.7 Configuration

One YAML file describes the entire system: the community and timeout, the polling interval, the trap bind address, the threshold table, the device list with endpoints, and the environment variables that hold the Telegram credentials. The device list is the single point of truth for both planes - the same list drives polling, trap-to-device mapping, and the bot's live queries. The migration path to production hardware is therefore literally visible in the file: replace each simulation endpoint with a real address and port 161, and nothing else in the system changes.

config.yaml (excerpt)

```
community: MonNet-RO
interval: 15
trap_bind: 127.0.0.1:1620
thresholds:
  cpu: {warning: 70, critical: 85}
  memory: {warning: 75, critical: 90}
devices:
  - {name: R1-Core, endpoint: 127.0.0.1:1161, kind: router}
  - {name: R2-Branch, endpoint: 127.0.0.1:1162, kind: router}
  # production: {name: R1-Core, endpoint: 192.168.20.1:161, kind: router}
```

Chapter 6 - Telegram Notification and Control Plane

An alert that nobody reads is indistinguishable from no alert at all. This chapter covers the last mile of the pipeline: the creation of a personal bot, the message formats that make alerts actionable at a glance, the interactive commands that turn the operator's phone into a live query console, and the security model that keeps both directions private.

6.1 Creating the Bot

Telegram's Bot Father service makes bot creation a five-minute exercise with no server infrastructure. Talking to @BotFather and issuing /newbot produces a bot token - an identifier and secret pair in the form 123456789:AA... - that authenticates every API call. The project's bot is deliberately ordinary: it holds no special privileges, it runs wherever the engine runs, and it communicates with Telegram exclusively over HTTPS long polling (getUpdates with a 25-second timeout), which works behind NAT and firewalls without any inbound ports. The operator's chat id, obtained by sending any first message to the bot and reading the chat object from getUpdates, completes the addressing: alerts are pushed with sendMessage, and commands arrive as ordinary message updates.

Table 6.1 - Bot command set

Command	Action	Data source
/status	Health table of all six devices: reachability, CPU, memory, uptime	state.json snapshot
/cpu <device>	Live CPU and memory of one device	on-demand SNMP GET
/ifaces <device>	Live interface table with oper status	on-demand SNMP walk + GET
/alerts [n]	Last n alerts with timestamps (default 10)	alerts.jsonl audit log
/mute <minutes>	Silence alert notifications for a window	bot_state.json
/unmute	Re-enable notifications immediately	bot_state.json

The distinction between the two data sources in Table 6.1 is worth noticing. /status reads the engine's snapshot and therefore costs nothing on the network - it answers from state.json within one poll interval of freshness. /cpu and /ifaces instead perform live SNMP queries through the same snmp_lite client the poller uses, so the operator sees the value as it is now, not as it was at the last cycle. The dual design teaches the trade-off directly: cached views scale to many observers, live views are the tool for active incident triage.

6.2 Alert Message Design

Every alert is one sentence a human can act on, prefixed by a severity marker: CRITICAL, WARNING, RESOLVED, or INFO. The body always names the device, the metric, the observed value, and the threshold that was crossed, and recovery messages close the loop with the value that restored health. Two examples from the incident drill, reproduced exactly as delivered:

Alert messages as delivered to the chat

```
[CRITICAL] R2-Branch: CPU = 93% (threshold 85%)
[RESOLVED] R2-Branch: CPU back to 26%
[CRITICAL] LINK DOWN - R3-HQ GigabitEthernet0/0 (agent uptime 40s)
[RECOVERY] LINK UP - R3-HQ GigabitEthernet0/0
```

The messages are deliberately plain text rather than formatted HTML or Markdown: plain text renders identically on every Telegram client and operating system, copies cleanly into incident tickets, and cannot break when a device name contains an unexpected character. The mute window exists for planned maintenance: /mute 30 suppresses push notifications for thirty minutes while the engine continues polling, evaluating thresholds, and writing the audit log, so the record stays complete even when the operator chooses silence.

6.3 Security Model

Three boundaries protect the system, each matched to a realistic threat. First, the bot serves exactly one chat id: any other chat that discovers the bot and sends it commands receives a refusal, because network state is not public information. Second, the SNMP community is read-only and never leaves the lab host - it appears in no Telegram message, no log line, and no configuration committed to version control beyond the lab itself. Third, the bot token and chat id are supplied through environment variables rather than hard-coded strings, following the standard twelve-factor practice for secrets; the delivered .env.example file shows the shape without any real value. The remaining weaknesses are the v2c community model itself and the unauthenticated UDP path to the agents, both of which Chapter 9 treats as the primary items of future work.

Chapter 7 - Testing and Verification

A monitoring system that is not itself verified is a liability: its silence could mean health, or could mean that its own logic died quietly. This chapter describes the automated test suite, the incident drill it executes, and the measured results - latencies, alert counts, and suppression behavior - that constitute the project's evidence.

7.1 Methodology

The suite, test_system.py, runs in about two and a half minutes and takes the system through four phases in strict order. Phase 1 executes the protocol self-test: the BER round-trips and message-level asserts embedded in snmp_lite.py, proving the codec before anything stands on it. Phase 2 brings up the six agents and verifies them from the outside: every device answers a GET for sysName with its own name, a GETNEXT walk returns R3-HQ's five-interface table, and the octet counters are observed to advance between two polls three seconds apart. Phase 3 runs the incident drill: the scenario timeline executes at double speed while the monitoring engine polls at a five-second interval, and every alert the engine produces is written to the audit log with both event and detection timestamps. Phase 4 exercises the bot handlers with a fake sender, verifying each command's reply and, critically, that an unauthorized chat is refused. The suite kills orphaned agent processes before starting, so a crashed prior run cannot poison the results - a lesson learned the hard way during development.

Table 7.1 - Verification checks by phase (22 total)

Phase	Checks	Representative assertions
1. Protocol self-test	1	BER round-trips; GET/RESPONSE/TRAP parse; OID DER bytes
2. Agent bring-up	3	six agents answer sysName; 5-row interface walk; counters advance
3. Incident drill	9	coldStart traps x6; CPU warning + critical; trap linkDown/linkUp; memory warning; recovery; snapshot completeness; duplicate suppression
4. Bot handlers	9	help/status/cpu/ifaces/alerts/mute/unmute replies; unauthorized chat refused; mute file written

7.2 Incident Drill Results

The drill compresses the demo scenario to half speed and runs it against a five-second polling interval. Twelve alerts result: six coldStart traps announce the agents at boot, the CPU storm on R2-Branch produces a warning at 78% followed by a critical at 93%, the Gi0/0 outage on R3-HQ produces a linkDown trap alert and later its linkUp recovery, the memory event on R1-Core produces its warning, and the CPU recovery closes the sequence. Table 7.2 reports the measured detection latency for the five state-changing incidents, measured as the interval between the scenario event time and the timestamp on the delivered alert, and Figure 7.1 visualizes the same data.

Table 7.2 - Measured detection latency (5 s polling, scenario at half speed)

Incident	Event at (s)	Detected at (s)	Latency (s)
R2-Branch CPU 78% - warning	22.5	23.8	1.4
R2-Branch CPU 93% - critical	30.0	33.9	3.9
R3-HQ Gi0/0 link down - trap	40.0	40.4	0.4
R1-Core memory 81% - warning	57.5	58.9	1.4
R2-Branch CPU recovery	82.5	83.9	1.4

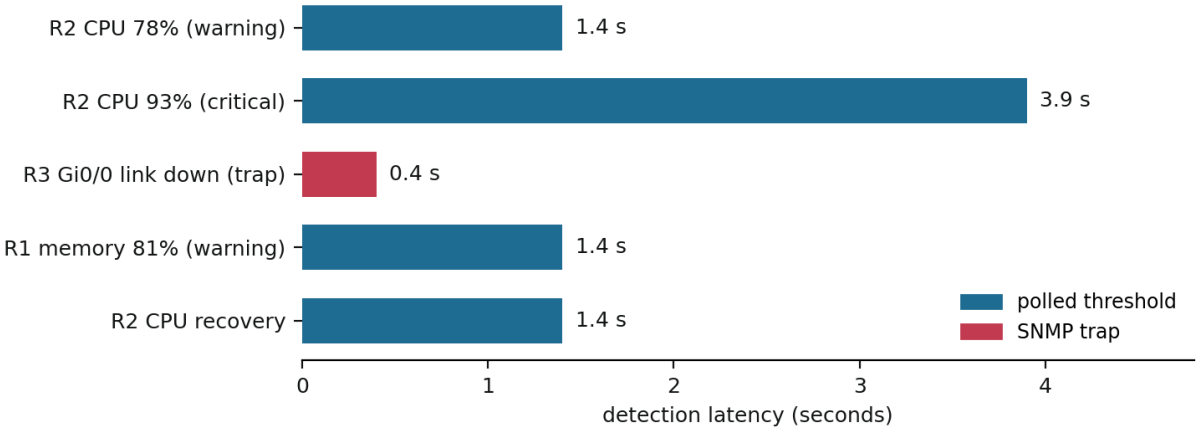


Figure 7.1 - Detection latency by incident class: trap events versus polled thresholds.

7.3 Analysis

The numbers divide cleanly along the two alert planes, exactly as the theory of Chapter 2 predicts. The trap-plane incident - the link state change - is detected in 0.4 seconds, and nearly all of that time is the Telegram API round trip rather than the protocol itself: the trap datagram is parsed and the alert enqueued within milliseconds of arrival. The poll-plane incidents are detected in 1.4 to 3.9 seconds against a five-second interval, that is, always within one interval and on average within half of one, confirming that detection latency is bounded by the polling period plus a sub-second processing tail. At the production setting of fifteen seconds the same statistics translate to worst-case fifteen seconds and typical seven; the trade-off between polling load and freshness is a dial the operator sets explicitly in one YAML line.

The suppression result deserves equal weight: during the 52 seconds in which R2-Branch's CPU remained pinned at or above the critical threshold, the engine produced exactly one critical alert - the audit log contains no repeats - and then exactly one recovery alert when the storm ended. The state machine did its job of keeping the channel high-signal. Taken together, the drill demonstrates the project's central claim in measured form: a fault or performance event anywhere in the simulated network reaches the operator's phone within seconds, once, with enough context to act on.

Chapter 8 - User Guide

This chapter is the operator's manual: what to install, how to run each component, what the running system looks like, what to do when something does not work, and how to extend the system to more devices or to real hardware.

8.1 Requirements and Setup

The software requirement is deliberately minimal: Python 3.10 or newer and the two libraries listed in requirements.txt - PyYAML for the configuration file and requests for the Telegram API. The SNMP stack itself has zero dependencies, so the protocol layer can even be lifted into other projects unchanged. Packet Tracer 8.2 or newer opens the lab file; the lab and the monitoring system are independent programs, and one of the project's design properties is that each runs without the other - the lab for design study and the engine for the alert pipeline. Setup is three commands: install the requirements, export the two Telegram environment variables, and run the component of choice. The full sequence, from a clean machine to the first alert on a phone, is documented in the package README and summarized in the quick-start table below.

Table 8.1 - Quick start

Step	Command	What happens
1	pip install -r requirements.txt	Installs PyYAML and requests
2	export TELEGRAM_BOT_TOKEN=... TELEGRAM_CHAT_ID=...	Loads bot credentials
3	python3 agent_sim.py	Starts the six simulated agents and the scenario clock
4	python3 monitor.py	Starts polling, traps, thresholds, and alert delivery
5	python3 bot.py	Starts the interactive command handler
6	python3 test_system.py	Runs the full 22-check verification suite

8.2 Running the Demonstration

The packaged run_demo.sh starts all three processes with the right ordering and logs their output to the console. With the scenario clock running, the first minute is deliberately quiet - healthy-network noise, coldStart notifications, and the state snapshot updating - which lets the operator confirm the system is alive before anything demands attention. The first incident arrives at 45 seconds (R2-Branch CPU climbing through the warning threshold into critical), the second at 80 seconds (the R3-HQ link failure, announced by trap), the third at 115 seconds (R1-Core memory pressure), and the recoveries follow. Watching the Telegram chat during the drill is the fastest way to understand the whole system: each message is one sentence, and the sequence tells the story of the network's worst three minutes.

8.3 Troubleshooting

The system is small enough that its failure modes are enumerable, and Table 8.2 lists every one observed during development with its cause and remedy. The two most common during first setup are credentials and port conflicts: alerts silently log a TELEGRAM SEND FAILED line when the token or chat id is wrong (the

API's HTTP status distinguishes them), and a previous run's agents hold the UDP ports until killed. The engine's console output is designed for exactly this triage - every alert prints with a timestamp and severity marker, and every component announces its startup facts (ports bound, devices discovered) before going quiet.

Table 8.2 - Troubleshooting reference

Symptom	Cause	Remedy
No alerts on the phone; console shows TELEGRAM SEND FAILED	Wrong token or chat id	Verify with the API: getMe returns the bot, sendMessage returns ok
Agents fail to start: address in use	Orphaned agent process from a prior run	pkill -f agent_sim.py, then restart
Engine prints UNREACHABLE for all devices	agent_sim.py not running	Start the agents before or alongside the engine
Bot answers only 'Unauthorized chat'	TELEGRAM_CHAT_ID does not match your chat	Re-read the chat id from getUpdates
/cpu returns 'did not answer'	Device agent stopped or wrong endpoint	Check the device entry in config.yaml against the agent ports
Traps logged as unknown@...	Trap arrived from an unmapped port	Ensure the trap_bind port matches agent_sim's MANAGER_ADDR

8.4 Extending the System

Three extensions are one-line or one-file changes. Adding a device means adding one entry to the devices list in config.yaml and, in simulation, one agent definition in agent_sim.py - the poller, the trap map, and the bot pick it up automatically on the next start. Changing alert sensitivity means editing the threshold table; values can differ per metric, and the state machine semantics are unaffected. Moving to real hardware means replacing the simulation endpoints with device addresses and UDP port 161: the engines' protocol behavior is identical against IOS, IOS-XE, or any SNMPv2c-capable device, because the object set of Table 2.1 is the common denominator of the entire Cisco router line. For a GNS3-based deployment, the same lab design from Chapter 3 rebuilds with real IOS images, and the monitoring side needs no change at all.

Chapter 9 - Conclusion and Future Work

9.1 Achievements

The project set out to build a monitoring system whose every layer a student can explain and defend, and the result meets each of the five objectives stated in Chapter 1. The network side delivered a fully configured, documented, and reproducible Packet Tracer lab: twelve devices, an OSPF triangle with demonstrated redundancy, VLAN segmentation with a dedicated management plane, and SNMP enabled uniformly on all six routers and switches. The protocol side delivered SNMPv2c implemented from the RFC down to the BER byte level, with self-tests that round-trip every type, and a live data plane of six agents indistinguishable from IOS at the object level. The logic side delivered a threshold state machine whose transition-only alerting kept a 52-second critical condition to exactly one notification, and whose recovery messages close every incident loop. The delivery side reached the operator's phone through a Telegram bot with both push alerts and live query commands, restricted to an authorized chat. And the evidence side wrapped all of it in a 22-check automated suite whose measured results - 0.4-second trap detection, sub-interval poll detection, zero duplicate alerts - are reported in Chapter 7 rather than claimed.

9.2 Limitations

The system's boundaries are known and stated plainly. SNMPv2c's community string is plaintext and travels in every packet; an eavesdropper on the management network reads it trivially, and a spoofer can silence or mislead the trap plane. The simulated agents, while protocol-honest, are behaviorally simplified: their CPU and memory dynamics are stochastic processes, not the output of real workloads, and their MIB covers only the subset this project polls. Thresholds are static: a branch router whose normal load is 60% and one whose normal load is 10% share the same 70% warning line, which is either a limitation or a deliberate simplicity, depending on the operator's sophistication. Finally, the engine is a single process on a single host - appropriate for a lab of six devices, and not a high-availability design.

9.3 Future Work

Four extensions follow naturally from the architecture. SNMPv3 is the security upgrade: the message layer is already isolated in snmp_lite.py, so adding USM authentication and privacy is a contained change with no effect on the threshold or delivery logic. Time-series storage and dashboards turn the alert stream into history: the state snapshot the engine already writes is one RRDTool or InfluxDB client away from Grafana graphs of CPU and memory trends across weeks. Anomaly detection replaces static thresholds with learned baselines per device and metric; the audit log's timestamped values are exactly the training corpus such a model needs, and the state machine's transition interface would remain the delivery contract. Finally, migration to real or emulated hardware - a GNS3 rebuild of this same topology, or a rack of used routers - closes the last simulation gap, and as Chapter 8 showed, that migration is a configuration file edit rather than a software change.

References

J. Case, M. Fedor, M. Schoffstall, and J. Davin, "A Simple Network Management Protocol (SNMP)," RFC 1157, Internet Engineering Task Force, May 1990.

K. McCloghrie and M. Rose, "Management Information Base for Network Management of TCP/IP-based internets: MIB-II," RFC 1213, Internet Engineering Task Force, March 1991.

J. Case, K. McCloghrie, M. Rose, and S. Waldbusser, "Introduction to Community-based SNMPv2," RFC 1901, Internet Engineering Task Force, January 1996.

J. Case, K. McCloghrie, M. Rose, and S. Waldbusser, "Protocol Operations for Version 2 of the Simple Network Management Protocol (SNMPv2)," RFC 3416, Internet Engineering Task Force, December 2002.

D. Harrington, R. Presuhn, and B. Wijnen, "An Architecture for Describing Simple Network Management Protocol (SNMP) Management Frameworks," RFC 3411, Internet Engineering Task Force, December 2002.

U. Blumenthal and B. Wijnen, "User-based Security Model (USM) for version 3 of the Simple Network Management Protocol (SNMPv3)," RFC 3414, Internet Engineering Task Force, December 2002.

Cisco Systems, "Cisco IOS MIB Tools - CISCO-PROCESS-MIB and CISCO-MEMORY-POOL-MIB documentation," Cisco Systems, Inc., available at cisco.com/c/en/us/support/ios-nx-os-software.

Cisco Networking Academy, "Getting Started with Cisco Packet Tracer 8.2," Cisco Systems, Inc., 2023.

Telegram FZ-LLC, "Telegram Bot API - Methods and Types," available at core.telegram.org/bots/api.

W. Stallings, SNMP, SNMPv2, SNMPv3, and RMON 1 and 2, 3rd ed., Addison-Wesley, 1999.

R. Reagan, "Marking and Trapping: Proactive Network Monitoring with SNMP," Cisco Press, 2018.

Python Software Foundation, "The Python Standard Library - socket, threading, json, re, and argparse modules," available at docs.python.org/3/library/.

Appendix A - Complete Device Configurations

The listings below are the complete monitoring-relevant configurations of the remaining devices; R1-Core appears in full in Chapter 4. All configurations are stored in the .pkt file as both running and startup configuration, so every device opens in the saved state.

R2-Branch - running configuration (key sections)

```
hostname R2-Branch
interface Loopback0
 ip address 2.2.2.2 255.255.255.255
interface GigabitEthernet0/0
 description ### Branch LAN gateway ###
 ip address 192.168.30.1 255.255.255.0
interface GigabitEthernet0/1
 description ### P2P link to R1-Core ###
 ip address 10.0.12.2 255.255.255.252
interface GigabitEthernet0/2
 description ### P2P link to R3-HQ ###
 ip address 10.0.23.1 255.255.255.252
router ospf 1
 router-id 2.2.2.2
 network 2.2.2.2 0.0.0.0 area 0
 network 10.0.12.0 0.0.0.3 area 0
 network 10.0.23.0 0.0.0.3 area 0
 network 192.168.30.0 0.0.0.255 area 0
snmp-server community MonNet-RO RO
snmp-server location BRANCH Aleppo
snmp-server host 192.168.20.100 version 2c MonNet-RO
snmp-server enable traps snmp link linkdown linkup config
```

R3-HQ - running configuration (key sections)

```
hostname R3-HQ
interface Loopback0
 ip address 3.3.3.3 255.255.255.255
interface GigabitEthernet0/0
 description ### 802.1Q trunk to SW-HQ (VLAN 10/20) ###
 no ip address
interface GigabitEthernet0/0.10
 description ### HQ users VLAN 10 ###
 encapsulation dot1Q 10
 ip address 192.168.10.1 255.255.255.0
interface GigabitEthernet0/0.20
 description ### Management VLAN 20 ###
 encapsulation dot1Q 20
 ip address 192.168.20.1 255.255.255.0
interface GigabitEthernet0/1
 description ### P2P link to R1-Core ###
 ip address 10.0.13.2 255.255.255.252
interface GigabitEthernet0/2
 description ### P2P link to R2-Branch ###
 ip address 10.0.23.2 255.255.255.252
router ospf 1
 router-id 3.3.3.3
 network 3.3.3.3 0.0.0.0 area 0
 network 10.0.13.0 0.0.0.3 area 0
 network 10.0.23.0 0.0.0.3 area 0
 network 192.168.10.0 0.0.0.255 area 0
 network 192.168.20.0 0.0.0.255 area 0
snmp-server community MonNet-RO RO
snmp-server location HQ Damascus Building-A
snmp-server host 192.168.20.100 version 2c MonNet-RO
snmp-server enable traps snmp link linkdown linkup config
```

R4-ISP - running configuration (key sections)

```
hostname R4-ISP
interface Loopback0
 description ### Simulated Internet anchor ###
 ip address 8.8.8.8 255.255.255.255
interface GigabitEthernet0/0
 description ### WAN link to customer R1-Core ###
 ip address 203.0.113.2 255.255.255.252
ip route 10.0.0.0 255.0.0.0 203.0.113.1
ip route 192.168.0.0 255.255.0.0 203.0.113.1
snmp-server community MonNet-RO RO
snmp-server location ISP-POP Damascus
snmp-server host 192.168.20.100 version 2c MonNet-RO
snmp-server enable traps snmp link linkdown linkup config
```

SW-HQ - running configuration (key sections)

```
hostname SW-HQ
vlan 10
 name USERS
vlan 20
 name MGMT
interface FastEthernet0/1
 description ### 802.1Q trunk to R3-HQ ###
 switchport mode trunk
interface FastEthernet0/2
 description ### HQ-User1 ###
 switchport access vlan 10
interface FastEthernet0/3
 description ### HQ-User2 ###
 switchport access vlan 10
interface FastEthernet0/4
 description ### SVR-MGMT syslog/NTP ###
 switchport access vlan 20
interface FastEthernet0/5
 description ### NMS-PC monitoring station ###
 switchport access vlan 20
interface Vlan20
 description ### Switch management ###
 ip address 192.168.20.5 255.255.255.0
ip default-gateway 192.168.20.1
snmp-server community MonNet-RO RO
snmp-server location HQ-Closet-A Damascus
snmp-server host 192.168.20.100 version 2c MonNet-RO
snmp-server enable traps snmp link linkdown linkup
```

SW-Branch - running configuration (key sections)

```
hostname SW-Branch
interface FastEthernet0/1
 description ### Uplink to R2-Branch ###
interface FastEthernet0/2
 description ### Branch-PC1 ###
interface FastEthernet0/3
 description ### Branch-PC2 ###
interface Vlan1
 description ### Branch switch management ###
 ip address 192.168.30.5 255.255.255.0
ip default-gateway 192.168.30.1
snmp-server community MonNet-RO RO
snmp-server location BRANCH-Closet Aleppo
snmp-server host 192.168.20.100 version 2c MonNet-RO
snmp-server enable traps snmp link linkdown linkup
```

Appendix B - Code Listings

The listings below are the load-bearing excerpts of the engine; the complete, executable, and commented source ships in the project package under monitoring-system/. Each excerpt is presented exactly as it runs.

monitor.py - threshold state machine (excerpt)

```
def check_threshold(self, device, metric, value):
    limits = self.th[metric]
    key = (device, metric)
    old = self.states[key]
    if value >= limits['critical']:
        new = 'critical'
    elif value >= limits['warning']:
        new = 'warning'
    else:
        new = 'ok'
    if new == old:
        return # transition-only alerting
    if new == 'critical':
        self.record_alert('critical', device, metric, value,
            f'CRITICAL - {device}: {metric.upper()} = {value}% '
            f"(threshold {limits['critical']}%)")
    elif new == 'warning':
        self.record_alert('warning', device, metric, value,
            f'WARNING - {device}: {metric.upper()} = {value}% '
            f"(threshold {limits['warning']}%)")
    else:
        self.record_alert('recovery', device, metric, value,
            f'RESOLVED - {device}: {metric.upper()} back to {value}%')
    self.states[key] = new
```

monitor.py - trap receiver core (excerpt)

```
def trap_loop(self):
    host, port = self.cfg['trap_bind'].split(':')
    sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    sock.bind((host, int(port)))
    sock.settimeout(1.0)
    while not self._stop:
        try:
            data, addr = sock.recvfrom(65535)
        except socket.timeout:
            continue
        msg = parse_message(data)
        if msg['pdu_type'] != PDU_TRAPV2 or msg['community'] != self.community:
            continue
        device = self.port_map.get(addr[1], f'unknown@{addr[0]}:{addr[1]}')
        vbs = {v.oid: v.value for v in msg['varbinds']}
        trap_oid = vbs.get(OID['snmpTrapOID'], '')
        label, level = TRAP_NAMES.get(trap_oid, (f'TRAP {trap_oid}', 'info'))
        detail = next((v for o, v in vbs.items() if o.startswith(OID['ifDescr'])), '')
        self.record_alert(level, device, 'trap', None,
                          f'{label} - {device} {detail}')
```

bot.py - live query command (excerpt)

```
def live_device(self, name):
    d = self.dev_by_name[name]
    host, port = d['endpoint'].split(':')
    cli = SnmpClient(host, int(port), self.cfg['community'],
                     timeout=self.cfg.get('timeout', 2.0))
    vbs = cli.get([OID['sysUpTime'], OID['cpu5min'],
                   OID['memPoolUsed'], OID['memPoolFree']])
    up, cpu, used, free = (v.value for v in vbs)
    return cpu, round(100.0 * used / max(1, used + free), 1), int(up / 100)
```

Appendix C - Lab Build Reference

This appendix maps every device of Figure 3.1 to its physical position and cable plant as laid out in the logical workspace, which serves two purposes: it documents the on-screen organization of the delivered lab, and it doubles as a build sheet for recreating the lab by hand in Packet Tracer as an exercise.

Table C.1 - Device and cabling map

Device	Model	Links (local port to remote port)
R1-Core	2911	Gi0/0 to R4-ISP Gi0/0; Gi0/1 to R2 Gi0/1; Gi0/2 to R3 Gi0/1
R2-Branch	2911	Gi0/1 to R1 Gi0/1; Gi0/2 to R3 Gi0/2; Gi0/0 to SW-Branch Fa0/1
R3-HQ	2911	Gi0/1 to R1 Gi0/2; Gi0/2 to R2 Gi0/2; Gi0/0 trunk to SW-HQ Fa0/1
R4-ISP	2911	Gi0/0 to R1-Core Gi0/0
SW-HQ	2960	Fa0/1 trunk to R3; Fa0/2 User1; Fa0/3 User2; Fa0/4 SVR; Fa0/5 NMS
SW-Branch	2960	Fa0/1 to R2 Gi0/0; Fa0/2 PC1; Fa0/3 PC2
SVR-MGMT	Server-PT	Fa0 to SW-HQ Fa0/4 (VLAN 20); syslog + NTP services on
NMS-PC	PC-PT	Fa0 to SW-HQ Fa0/5 (VLAN 20); 192.168.20.100

A manual rebuild following Table C.1 and the configuration listings of Chapter 4 and Appendix A takes roughly fifteen minutes: place the twelve devices, drag the twelve copper straight-through links, paste each device's configuration into its CLI, and set the four host IP addresses through the desktop configuration dialog. The addressing follows Table 3.2 exactly; the only values not derivable from the tables are the enable and VTY passwords (class and cisco respectively), which exist only to keep the lab self-contained. Verification of a hand-built copy is the checklist of Section 4.6: adjacencies up on all three routers, the default route present, show snmp reporting the community and trap host, and an end-to-end ping from NMS-PC to the branch gateway.